

Apuntes 4 — IC-7602 Redes

Estudiante: Joshua Obando Castro **Carné:** 2023130774

Fecha: 2026-05-22 **Profesor:** Nereo Campos

1. Cómo se estructura Internet

Internet es básicamente un conjunto de sistemas autónomos interconectados, donde distintos actores como cloud providers, ISPs y gobiernos tienen su propio pedazo de red y se conectan entre sí para mover tráfico. En ese ecosistema existen los Backbones o redes de nivel 1, que son redes enormes prácticamente a prueba de fallos y por donde pasa la mayor cantidad del tráfico de Internet. Miami, por ejemplo, es uno de los puntos de convergencia más grandes de la región, y es muy probable que gran parte del tráfico de una videollamada en Costa Rica esté pasando por ahí.

Cuando una máquina en AWS quiere comunicarse con un servidor en GCP, el paquete sale por el Internet Gateway de AWS, se routea a través de Internet usando protocolos como OSPF o BGP, llega a GCP y recorre su red interna hasta el servidor destino. El camino de regreso puede tomar una ruta completamente diferente. Internet en este sentido funciona como un grafo gigante y cambiante, donde los paquetes van buscando su camino salto a salto.

Una característica fundamental de Internet es que es un medio de **mejor esfuerzo**: no promete entregar los paquetes, simplemente lo intenta. El tamaño máximo de un paquete a nivel de Ethernet es de 1500 bytes, aunque capas superiores pueden manejar hasta 64K. Esto significa que paquetes grandes tienen que fragmentarse para poder moverse, lo que genera procesamiento adicional en cada salto de la red.

2. VPC — Virtual Private Cloud

Cuando uno crea recursos en cualquier cloud provider, siempre tiene que hacerlo dentro de una red virtual privada. En AWS eso se llama VPC (Virtual Private Cloud), en GCP se llama Virtual Private Cloud igual, y en Azure se llama VNet (Virtual Network). El concepto es el mismo en todos: es el contenedor lógico de red dentro del cual viven los recursos de una cuenta.

La ventaja principal de tener una VPC es que todo el tráfico entre servicios dentro de la misma cuenta se resuelve localmente, sin salir a Internet, lo que hace que la latencia sea órdenes de magnitud menor que si hubiera que dar una vuelta por la red pública. El tráfico que viaja de un servicio a otro dentro del VPC puede rondar los 10 milisegundos, mientras que salir a Internet puede elevar ese número a 80 o 100 milisegundos fácilmente, aparte de que los cloud providers cobran por el tráfico que sale de la red.

Sin embargo, hay servicios que no viven dentro del VPC de una cuenta, como blob storage (S3 en AWS, Blob Storage en Azure, Cloud Storage en GCP). Cuando una máquina dentro del VPC necesita comunicarse con uno de estos servicios, por defecto la petición sale a Internet, da una vuelta por la red pública del cloud provider y regresa, lo que incrementa la latencia y genera costos de transferencia de datos.

2.1 VPC Endpoints

La solución para ese problema son los **VPC Endpoints**, que permiten que el tráfico hacia servicios externos al VPC se mantenga dentro de la red privada del cloud provider sin salir a Internet. Lo que hacen es conectarse a la Route Table del VPC, de modo que cuando una petición va hacia, por ejemplo, un bucket de S3, antes de que salga por el Internet Gateway el Route Table la intercepta y la manda a través del Endpoint directamente, sin tocar Internet. Esto resulta aproximadamente un 50% más rápido que ir por Internet, elimina los costos de transferencia y además mejora la privacidad porque el tráfico nunca viaja por redes públicas.

3. VPC Peering y Zero Trust Architecture

Cuando se quiere conectar dos VPCs entre sí, ya sea de la misma cuenta o de cuentas distintas, el mecanismo más directo es el **VPC Peering**. Esto establece una relación de confianza entre las dos redes a nivel de capa 3 y 4, lo que permite que el tráfico se rutee entre ellas a través de la Route Table sin salir a Internet. Es útil por ejemplo cuando una empresa quiere vender un servicio a otro cliente sin que el tráfico salga a la red pública.

El problema del VPC Peering es que al abrir las dos redes entre sí, se expone todo el rango de IPs de ambas redes. Eso significa que alguien con acceso malicioso dentro de uno de los VPCs podría usar herramientas como Nmap para escanear toda la red del otro lado y detectar vulnerabilidades. Esto va en contra del principio de **Zero Trust Architecture**, que básicamente dice que no se debe confiar en nadie ni en nada a nivel de red, sin importar si está dentro o fuera del perímetro.

3.1 PrivateLink

La solución más robusta para hacer interconexión de redes sin sacrificar seguridad es el **PrivateLink**, disponible en todos los cloud providers. En lugar de abrir las dos redes completas como hace el VPC Peering, PrivateLink expone únicamente un punto de entrada privado dentro del VPC consumidor, que se mapea exclusivamente al recurso específico que se quiere acceder en el otro VPC. Entonces la comunicación puede restringirse a un protocolo específico, por ejemplo solo HTTPS por el puerto 443, y además se le puede aplicar un WAF (Web Application Firewall) en ese punto de entrada. Esto hace que aunque se conecten dos VPCs, el acceso sea exactamente lo que se necesita y nada más, cumpliendo con Zero Trust.

4. Serverless y contenedores en Cloud

En el mundo del cloud hoy en día los dos paradigmas más dominantes son los **contenedores** y el **serverless**. Los contenedores pueden correr en distintos motores dependiendo del cloud provider.

Motor	AWS	GCP	Azure
Contenedores sin Kubernetes	ECS (Elastic Container Service)	Cloud Run	Azure Container Apps
Kubernetes gestionado	EKS (Elastic Kubernetes Service)	GKE (Google Kubernetes Engine)	AKS (Azure Kubernetes Service)
Funciones serverless	Lambda	Cloud Functions	Azure Functions

El concepto de **serverless** no significa que no haya servidor, sino que el servidor no vive en la cuenta del cliente. Amazon, por ejemplo, mantiene una flotilla interna de servidores dedicados a ejecutar funciones

Lambda. Cuando alguien crea una Lambda Function, Amazon toma el paquete de ejecución, lo carga en uno de esos servidores, lo ejecuta y devuelve el resultado. Lo que hace que esto funcione a nivel de red es que cada vez que se ejecuta algo serverless, se crea una tarjeta de red virtual llamada ENI (Elastic Network Interface) que agarra un IP interno del VPC del cliente, lo que le da conectividad directa hacia los recursos de esa cuenta.

5. Diseño de red y dimensionamiento de IPs

Este es uno de los temas donde el conocimiento de redes tiene impacto directo en el trabajo del día a día como ingeniero de software. Cuando se trabaja con contenedores o serverless a escala, cada Pod en Kubernetes y cada Task en ECS consume un IP interno del VPC. Eso significa que si el diseño de la subred no tiene suficientes IPs disponibles, el sistema simplemente no puede levantar toda la capacidad que se pagó.

Un ejemplo concreto: si se configura una función Lambda con una concurrencia máxima de 400 ejecuciones simultáneas, pero la subred tiene máscara /24 y por lo tanto solo 254 IPs disponibles, en el pico de carga el sistema no va a poder ejecutar todas las Lambdas porque la red se queda sin IPs. Eso es plata pagada que no se puede usar, y la causa es un mal diseño de red.

Lo mismo pasa con Kubernetes. Si un cluster de 20 máquinas tiene un MaxPods de 110 por nodo, eso implica hasta 2200 pods simultáneos, cada uno consumiendo un IP. Una subred /24 con 254 IPs no alcanza para nada. La máscara correcta en ese caso sería /20, que da 4094 IPs usables y sí puede soportar esa carga.

La regla general es: si se va a trabajar con microservicios, contenedores o serverless con alto volumen de peticiones, hay que planear la capacidad de IPs antes de crear la red, porque una vez que está creada y en producción, cambiarle la máscara es complicado.

6. Formato del paquete IPv4

El encabezado de IPv4 tiene 5 palabras mandatorias de 32 bits cada una, más opciones adicionales. Los campos más relevantes son:

El campo de **tipo de servicio** permite priorizar tráfico en los routers, por ejemplo darle prioridad a los paquetes de una llamada VoIP sobre los de un correo electrónico, ya que VoIP es tiempo real y el correo no lo es por naturaleza.

El flag **DF (Don't Fragment)** indica que el paquete no debe fragmentarse. Si un router necesita fragmentarlo pero tiene este flag activo, lo descarta.

El flag **More Fragments** indica que el paquete es parte de una secuencia y que todavía hay más fragmentos en camino, entonces el receptor debe esperar antes de rearmar el mensaje.

El **Fragment Offset** o desplazamiento del fragmento indica en qué posición dentro del paquete original va ese fragmento, lo que permite al receptor reordenarlos correctamente aunque lleguen en desorden.

El **Time to Live (TTL)** define cuántos saltos puede dar el paquete antes de ser descartado. Cada router que procesa el paquete decrementa este valor en uno, y cuando llega a cero lo elimina. Esto evita que paquetes perdidos queden circulando indefinidamente por Internet.

7. Direccionamiento IP y CIDR

Las direcciones IPv4 son de 32 bits y se componen de dos partes: la porción de red y la porción de host. La máscara de subred define cuántos bits corresponden a cada parte. Para determinar si dos IPs pertenecen a la misma red, se hace un AND entre la dirección y la máscara: si el resultado (número de red) es el mismo para ambas, están en la misma red.

El **CIDR Block** (Classless Interdomain Routing) es el concepto que usan todos los cloud providers para definir el rango de IPs de un VPC. Cuando se crea un VPC con CIDR Block 10.0.0.0/20, eso significa que ese VPC puede tener hasta 4096 IPs como máximo, y todas las subredes que se creen dentro de ese VPC deben estar contenidas dentro de ese rango y no pueden ser más grandes que él.

La institución **ICANN** es la que asigna bloques de IPs a diferentes regiones y proveedores de servicio. Costa Rica, por ejemplo, tiene su propio rango de IPs asignado por el ICANN a los ISPs del país.

Algunas direcciones especiales importantes son el **127.0.0.1** (loopback), que es el identificador de la misma máquina y que no puede usarse para comunicarse con otra, y la dirección de **broadcast**, que se forma poniendo todos los bits de host en 1 y se usa para enviar un mensaje a todos los hosts de una red.

8. Protocolos de la capa de red

ICMP (Internet Control Message Protocol): permite hacer verificaciones del estado de la red. El comando ping funciona a nivel de capa 3 usando ICMP, y traceroute también lo usa para mostrar los saltos que da un paquete por Internet.

ARP (Address Resolution Protocol): permite identificar la dirección física (MAC address) de una computadora a partir de su IP, lo que es necesario para la comunicación dentro de una misma red local.

DHCP (Dynamic Host Configuration Protocol): es el servicio que asigna automáticamente IPs a los dispositivos cuando se conectan a una red. Todos los routers domésticos lo tienen activo, y en la práctica ya casi no se configura manualmente porque los cloud providers y los ISPs lo manejan de forma transparente.

NAT (Network Address Translation): cuando un dispositivo en una red privada necesita salir a Internet, el router le presta su IP pública para hacer la petición, guarda en una tabla a quién corresponde cada conexión y al recibir la respuesta la entrega al dispositivo correcto. Es el equivalente a usar un casillero en Estados Unidos para recibir compras desde Costa Rica: la dirección que ve el vendedor es la del casillero, no la de la casa.